Journal of Computational Physics 335 (2017) 736–746

Contents lists available at ScienceDirect

Journal of Computational Physics

www.elsevier.com/locate/jcp

Inferring solutions of differential equations using noisy multi-fidelity data

Maziar Raissi [a], Paris Perdikaris [b], George Em Karniadakis [a,*]

[a] Division of Applied Mathematics, Brown University, Providence, RI, USA
[b] Department of Mechanical Engineering, Massachusetts Institute of Technology, Cambridge, MA, USA

ARTICLE INFO

Article history:
Received 17 July 2016
Received in revised form 21 January 2017
Accepted 25 January 2017
Available online 2 February 2017

Keywords:
Machine learning
Integro-differential equations
Multi-fidelity modeling
Uncertainty quantification

ABSTRACT

For more than two centuries, solutions of differential equations have been obtained either analytically or numerically based on typically well-behaved forcing and boundary conditions for well-posed problems. We are changing this paradigm in a fundamental way by establishing an interface between probabilistic machine learning and differential equations. We develop data-driven algorithms for general linear equations using Gaussian process priors tailored to the corresponding integro-differential operators. The only observables are scarce noisy multi-fidelity data for the forcing and solution that are not required to reside on the domain boundary. The resulting predictive posterior distributions quantify uncertainty and naturally lead to adaptive solution refinement via active learning. This general framework circumvents the tyranny of numerical discretization as well as the consistency and stability issues of time-integration, and is scalable to high-dimensions.
© 2017 Elsevier Inc. All rights reserved.

1. Introduction

Nearly two decades ago a visionary treatise by David Mumford anticipated that stochastic methods will transform pure and applied mathematics in the beginning of the third millennium, as probability and statistics will come to be viewed as the natural tools to use in mathematical as well as scientific modeling [1]. Indeed, in recent years we have been witnessing the emergence of a data-driven era in which probability and statistics have been the focal point in the development of disruptive technologies such as probabilistic machine learning [2,3]. Only to verify Mumford's predictions, this wave of change is steadily propagating into applied mathematics, giving rise to novel probabilistic interpretations of classical deterministic scientific methods and algorithms. This new viewpoint offers an elegant path to generalization and enables computing with probability distributions rather than solely relying on deterministic thinking. In particular, in the area of numerical analysis and scientific computing, the first hints of this paradigm shift were clearly manifested in the thought-provoking work of Diaconis [4], tracing back to Poincaré's courses on probability theory [5]. This line of work has recently inspired resurgence in probabilistic methods and algorithms [6–8] that offer a principled and robust handling of uncertainty due to model inadequacy, parametric uncertainties, and numerical discretization/truncation errors. In particular, several statistical inference techniques have been reported in [9–12] for constructing probabilistic time-stepping schemes for systems of ordinary differential equations (e.g., systems arising after a partial differential equation is discretized in space). In the same spirit, the work of [13–16] has highlighted the possibility of solving linear partial differential equations and quantifying parameter and

* Corresponding author.
E-mail addresses: maziar_raissi@brown.edu (M. Raissi), george_karniadakis@brown.edu (G.E. Karniadakis).

http://dx.doi.org/10.1016/j.jcp.2017.01.060
0021-9991/© 2017 Elsevier Inc. All rights reserved.

discretization uncertainty using Gaussian process priors. These developments are defining a new area of scientific research in which probabilistic machine learning and classical scientific computing coexist in unison, providing a flexible and general platform for Bayesian reasoning and computation. In this work, we exploit this interface by developing a novel Bayesian inference framework that enables learning from (noisy) data and equations in a synergistic fashion.

2. Problem setup

We consider general linear integro-differential equations of the form

$$\mathcal{L}_x u(x) = f(x), \tag{1}$$

where x is a D-dimensional vector that includes spatial or temporal coordinates, $\mathcal{L}_x$ is a linear operator, $u(x)$ denotes an unknown solution to the equation, and $f(x)$ represents the external force that drives the system. We assume that $f_L := f$ is a complex, expensive to evaluate, "black-box" function. For instance, f_L could represent force acting upon a physical system, the outcome of a costly experiment, the output of an expensive computer code, or any other unknown function. We assume limited availability of high-fidelity data for f_L, denoted by $\{\boldsymbol{x}_L, \boldsymbol{y}_L\}$, that could be corrupted by noise $\boldsymbol{\epsilon}_L$, i.e., $\boldsymbol{y}_L = f_L(\boldsymbol{x}_L) + \boldsymbol{\epsilon}_L$. In many cases, we may also have access to supplementary sets of less accurate models $f_\ell,\ \ell = 1, \ldots, L-1$, sorted by increasing level of fidelity, and generating data $\{\boldsymbol{x}_\ell, \boldsymbol{y}_\ell\}$ that could also be contaminated by noise $\boldsymbol{\epsilon}_\ell$, i.e., $\boldsymbol{y}_\ell = f_\ell(\boldsymbol{x}_\ell) + \boldsymbol{\epsilon}_\ell$. Such data may come from simplified computer models, inexpensive sensors, or uncalibrated measurements. In addition, we also have a small set of data on the solution u, denoted by $\{\boldsymbol{x}_0, \boldsymbol{y}_0\}$, perturbed by noise $\boldsymbol{\epsilon}_0$, i.e., $\boldsymbol{y}_0 = u(\boldsymbol{x}_0) + \boldsymbol{\epsilon}_0$, sampled at scattered spatio-temporal locations, which we call anchor points to distinguish them from boundary or initial values. Although they could be located on the domain boundaries as in the classical setting, this is not a requirement in the current framework as solution data could be partially available on the boundary or in the interior of either spatial or temporal domains. Here, we are not primarily interested in estimating f. We are interested in estimating the unknown solution u that is related to f through the linear operator $\mathcal{L}_x$. For example, consider a bridge subject to environmental loading. In a two-level of fidelity setting (i.e., $L = 2$), suppose that one could only afford to collect scarce but accurate (high-fidelity) measurements of the wind force $f_2(x)$ acting upon the bridge at some locations. In addition, one could also gather samples by probing a cheaper but inaccurate (low-fidelity) wind prediction model $f_1(x)$ at some other locations. How could this noisy data be combined to accurately estimate the bridge displacements $u(x)$ under the laws of linear elasticity? What is the uncertainty/error associated with this estimation? How can we best improve that estimation if we can afford another observation of the wind force? Quoting Diaconis [4], "once we allow that we don't know f, but do know some things, it becomes natural to take a Bayesian approach".

3. Solution methodology

The basic building blocks of the Bayesian approach adopted here are Gaussian process (GP) regression [17,18] and auto-regressive stochastic schemes [19,21]. This choice is motivated by the Bayesian non-parametric nature of GPs, their analytical tractability properties, and their natural extension to the multi-fidelity settings that are fundamental to this work. In particular, GPs provide a flexible prior distribution over functions, and, more importantly, a fully probabilistic workflow that returns robust posterior variance estimates which enable adaptive refinement and active learning [22–24]. The framework we propose is summarized in Fig. 1 and is outlined in the following.

Inspired by [19,21], we will present the framework considering two-levels of fidelity (i.e. $L = 2$), although generalization to multiple levels is straightforward. Let us start with the auto-regressive model $u(x) = \rho u_1(x) + \delta_2(x)$, where $\delta_2(x)$ and $u_1(x)$ are two independent Gaussian processes [17–19,21] with $\delta_2(x) \sim \mathcal{GP}(0, g_2(x, x'; \theta_2))$ and $u_1(x) \sim \mathcal{GP}(0, g_1(x, x'; \theta_1))$. Here, $g_1(x, x'; \theta_1), g_2(x, x'; \theta_2)$ are covariance functions, θ_1, θ_2 denote their hyper-parameters, and ρ is a cross-correlation parameter to be learned from the data (see Sec. 3.1). Then, one can trivially obtain

$$u(x) \sim \mathcal{GP}(0, g(x, x'; \theta)), \tag{2}$$

with $g(x, x'; \theta) = \rho^2 g_1(x, x'; \theta_1) + g_2(x, x'; \theta_2)$, and $\theta = (\theta_1, \theta_2, \rho)$. The key observation here is that the derivatives and integrals of a Gaussian process are still Gaussian processes. Therefore, given that the operator $\mathcal{L}_x$ is linear, we obtain

$$f(x) \sim \mathcal{GP}(0, k(x, x'; \theta)), \tag{3}$$

with

$$k(x, x'; \theta) = \mathcal{L}_x \mathcal{L}_{x'} g(x, x'; \theta). \tag{4}$$

Similarly, we arrive at the auto-regressive structure $f(x) = \rho f_1(x) + \gamma_2(x)$ on the forcing, where $\gamma_2(x) = \mathcal{L}_x \delta_2(x)$, and $f_1(x) = \mathcal{L}_x u_1(x)$ are consequently two independent Gaussian processes with $\gamma_2(x) \sim \mathcal{GP}(0, k_2(x, x'; \theta_2))$, $f_1(x) \sim \mathcal{GP}(0, k_1(x, x'; \theta_1))$. Furthermore, for $\ell = 1, 2$, $k_\ell(x, x'; \theta_\ell) = \mathcal{L}_x \mathcal{L}_{x'} g_\ell(x, x'; \theta_\ell)$.

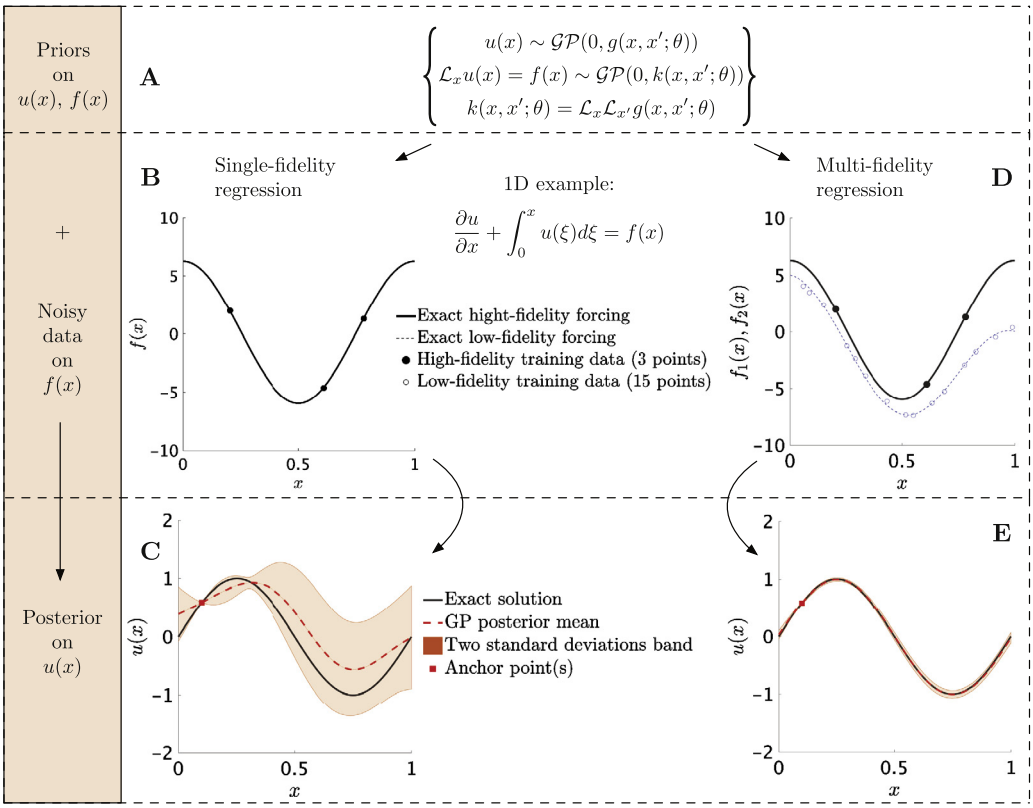

Fig. 1. *Inferring solutions of differential equations using noisy multi-fidelity data*: (A) Starting from a GP prior on u with kernel $g(x, x'; \theta)$, and using the linearity of the operator $\mathcal{L}_x$, we obtain a GP prior on f that encodes the structure of the differential equation in its covariance kernel $k(x, x'; \theta)$. (B) In view of 3 noisy high-fidelity data points for f, we train a single-fidelity GP (i.e., $\rho = 0$) with kernel $k(x, x'; \theta)$ to estimate the hyper-parameters θ. (C) This leads to a predictive posterior distribution for u conditioned on the available data on f and the anchor point(s) on u. For instance, in the one-dimensional integro-differential example considered here, the posterior mean gives us an estimate of the solution u while the posterior variance quantifies uncertainty in our predictions. (D), (E) Adding a supplementary set of 15 noisy low-fidelity data points for f, and training a multi-fidelity GP, we obtain significantly more accurate solutions with a tighter uncertainty band.

3.1. Training

The hyper-parameters $\theta = (\theta_1, \theta_2, \rho)$ which are shared between the kernels $g(x, x'; \theta)$ and $k(x, x'; \theta)$ can be estimated by minimizing the negative log marginal likelihood

$$\mathcal{NLML}(\theta, \sigma_{n_0}^2, \sigma_{n_1}^2, \sigma_{n_2}^2) := -\log p(\boldsymbol{y}|\boldsymbol{x}; \theta, \sigma_{n_0}^2, \sigma_{n_1}^2, \sigma_{n_2}^2), \tag{5}$$

with $\boldsymbol{y}^T := [\boldsymbol{y}_0^T\ \boldsymbol{y}_1^T\ \boldsymbol{y}_2^T]$ and $\boldsymbol{x}^T := [\boldsymbol{x}_0^T\ \boldsymbol{x}_1^T\ \boldsymbol{x}_2^T]$. Also, the variance parameters associated with the observation noise in $u(x)$, $f_1(x)$ and $f_2(x)$ are denoted by $\sigma_{n_0}^2, \sigma_{n_1}^2$, and $\sigma_{n_2}^2$, respectively. Finally, the negative log marginal likelihood is explicitly given by

$$\mathcal{NLML} = \frac{1}{2}\boldsymbol{y}^T K^{-1} \boldsymbol{y} + \frac{1}{2}\log|K| + \frac{n}{2}\log(2\pi), \tag{6}$$

where $n = n_0 + n_1 + n_2$, denotes the total number of data points in $\boldsymbol{x}^T := [\boldsymbol{x}_0^T\ \boldsymbol{x}_1^T\ \boldsymbol{x}_2^T]$, and

$$K = \begin{bmatrix} K_{00} & K_{01} & K_{02} \\ K_{10} & K_{11} & K_{12} \\ K_{20} & K_{21} & K_{22} \end{bmatrix}, \tag{7}$$

and

$$K_{00} = g(\boldsymbol{x}_0, \boldsymbol{x}_0; \theta) + \sigma_{n_0}^2 I_0, \tag{8}$$

$$K_{01} = K_{10}^T = \rho \mathcal{L}_{x'} g_1(\boldsymbol{x}_0, \boldsymbol{x}_1; \theta_1), \tag{9}$$

$$K_{02} = K_{20}^T = \mathcal{L}_{x'} g(\boldsymbol{x}_0, \boldsymbol{x}_2; \theta_2), \tag{10}$$

$$K_{11} = k_1(\boldsymbol{x}_1, \boldsymbol{x}_1; \theta_1) + \sigma_{n_1}^2 I_1, \tag{11}$$

$$K_{12} = K_{21}^T = \rho k_1(\boldsymbol{x}_1, \boldsymbol{x}_2; \theta_1), \tag{12}$$

$$K_{22} = k(\boldsymbol{x}_2, \boldsymbol{x}_2; \theta) + \sigma_{n_2}^2 I_2, \tag{13}$$

with I_0, I_1, and I_2 being the identity matrices of size n_0, n_1, and n_2, respectively.

3.2. Kernels

Without loss of generality, all Gaussian process priors used in this work are assumed to have zero mean and a squared exponential covariance function [17–19,21]. Moreover, anisotropy across input dimensions is handled by Automatic Relevance Determination (ARD) weights $w_{d,\ell}$ [17]

$$g_\ell(x, x'; \theta_\ell) = \sigma_\ell^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{D} w_{d,\ell}(x_d - x'_d)^2\right), \quad \text{for } \ell = 1, 2, \tag{14}$$

where σ_ℓ^2 is a variance parameter, x is a D-dimensional vector that includes spatial or temporal coordinates, and $\theta_\ell = \left(\sigma_\ell^2, \left(w_{d,\ell}\right)_{d=1}^{D}\right)$. The choice of the kernel represents our prior belief about the properties of the solutions we are trying to approximate. From a theoretical point of view, each kernel gives rise to a Reproducing Kernel Hilbert Space [17] that defines the class of functions that can be represented by our prior. In particular, the squared exponential covariance function chosen above, implies that we are seeking smooth approximations. More complex function classes can be accommodated by appropriately choosing kernels.

3.3. Cross-correlation parameter

If the training procedure yields a ρ close to zero, this indicates a negligible cross-correlation between the low- and high-fidelity data. Essentially, this implies that the low-fidelity data is not informative, and the algorithm will automatically ignore them, thus solely trusting the high-fidelity data. In general, ρ could depend on x (i.e., $\rho(x)$), yielding a more expressive scheme that can capture increasingly complex cross correlations [20,21]. However, for the sake of clarity, this is not pursued in this work.

3.4. Prediction

After training the model on data $\{\boldsymbol{x}_2, \boldsymbol{y}_2\}$ on f_2, $\{\boldsymbol{x}_1, \boldsymbol{y}_1\}$ on f_1, and anchor points data $\{\boldsymbol{x}_0, \boldsymbol{y}_0\}$ on u, we are interested in predicting the value of $u(x)$ at a new test point x. Hence, we need to compute the posterior distribution $p(u(x)|\boldsymbol{y})$. This can be achieved by first observing that

$$\begin{bmatrix} u(x) \\ \boldsymbol{y} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ \boldsymbol{0} \end{bmatrix}, \begin{bmatrix} g(x,x) & \boldsymbol{a} \\ \boldsymbol{a}^T & K \end{bmatrix}\right), \tag{15}$$

where $\boldsymbol{a}^T := [g(x, \boldsymbol{x}_0; \theta),\ \rho\mathcal{L}_{x'} g_1(x, \boldsymbol{x}_1; \theta_1),\ \mathcal{L}_{x'} g(x, \boldsymbol{x}_2; \theta)]$. Therefore, we obtain the predictive distribution $p(u(x)|\boldsymbol{y}) = \mathcal{N}\left(\overline{u}(x), V_u(x)\right)$, with predictive mean $\overline{u}(x) := \boldsymbol{a}^T K^{-1} \boldsymbol{y}$ and predictive variance $V_u(x) := g(x,x) - \boldsymbol{a}^T K^{-1} \boldsymbol{a}$.

3.5. Computational cost

The training step scales as $\mathcal{O}(mn^3)$, where m is the number of optimization iterations needed. In our implementation, we have derived the gradients of the likelihood with respect to all unknown parameters and hyper-parameters [17], and used a Quasi-Newton optimizer L-BFGS [33] with randomized initial guesses. Although this scaling is a well-known limitation of Gaussian process regression, we must emphasize that it has been effectively addressed by the recent works of Snelson & Gharhamani, and Hensman & Lawrence [34,35], and by the recursive multi-fidelity scheme put forth by Le Gratiet and Garnier [21]. Finally, we employ $\overline{u}(x)$ to predict $u(x)$ at a new test point x with a linear cost.

3.6. Adaptive refinement via active learning

Here we provide details on adaptive acquisition of data in order to enhance our knowledge about the solution u, under the assumption that we can afford one additional high-fidelity observation of the right-hand-side forcing f. The adaptation of the following active learning scheme to cases where one can acquire additional anchor points or low-fidelity data for f_1 is straightforward. We start by obtaining the following predictive distribution for $f(x)$ at a new test point x, $p(f(x)|\boldsymbol{y}) = \mathcal{N}(\overline{f}(x), V_f(x))$, where $\overline{f}(x) = \boldsymbol{b}^T K^{-1} \boldsymbol{y}$, $V_f(x) = k(x,x) - \boldsymbol{b}^T K^{-1} \boldsymbol{b}$, and $\boldsymbol{b}^T := [\mathcal{L}_x g(x, \boldsymbol{x}_0; \theta),\ \rho\mathcal{L}_x g_1(x, \boldsymbol{x}_1; \theta_1),\ k(x, \boldsymbol{x}_2; \theta)]$. The most intuitive sampling strategy corresponds to adding a new observation x^* for f at the location where the posterior variance is maximized, i.e.,

Table 1
Integro-differential equation in 1D: Optimized hyper-parameters and parameters – Single-fidelity case (i.e. $L = 1$).

σ_1^2	$w_{1,1}$	$\sigma_{n_1}^2$	$\sigma_{n_0}^2$
0.4132	21.787	1.5293e−07	3.3667e−10

Table 2
Integro-differential equation in 1D: Optimized hyper-parameters and parameters – Multi-fidelity case (i.e., $L = 2$).

σ_2^2	$w_{1,2}$	$\sigma_{n_2}^2$	$\sigma_{n_0}^2$
14.017	0.85152	5.3243e−07	9.991e−07
σ_1^2	$w_{1,1}$	$\sigma_{n_1}^2$	ρ
2.6028	6.3278	0.025547	1.3076

$$x^* = \arg\max_x V_f(x). \tag{16}$$

Compared to more sophisticated data acquisition criteria [22–24], we found that this simple computationally inexpensive choice leads to similar performance for all cases examined in this work. Designing the optimal data acquisition policy for a given problem is still an open question [22–24].

4. Results

4.1. Integro-differential equation in 1D

We start with a pedagogical example involving the following one dimensional integro-differential equation

$$\frac{\partial}{\partial x}u(x) + \int_0^x u(\xi)d\xi = f(x), \tag{17}$$

and assume that the low- and high-fidelity training data $\{\boldsymbol{x}_1, \boldsymbol{y}_1\}, \{\boldsymbol{x}_2, \boldsymbol{y}_2\}$ are generated according to $\boldsymbol{y}_\ell = f_\ell(\boldsymbol{x}_\ell) + \boldsymbol{\epsilon}_\ell$, $\ell = 1, 2$, where $\boldsymbol{\epsilon}_1 \sim \mathcal{N}(0, 0.3I)$, $\boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, 0.05I)$, $f_2(x) = 2\pi\cos(2\pi x) + \frac{1}{\pi}\sin(\pi x)^2$, and $f_1(x) = 0.8 f_2(x) - 5x$. This induces a non-trivial cross-correlation structure between $f_1(x), f_2(x)$. The training data points $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ are randomly chosen in the interval $[0, 1]$ according to a uniform distribution. Here we take $n_1 = 15$ and $n_2 = 3$, where n_1 and n_2 denote the sample sizes of $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$, respectively. Moreover, we have access to anchor point data $\{x_0, u_0\}$ on $u(x)$. For this example, we randomly selected x_0 in the interval $[0, 1]$ and let $y_0 = u(x_0)$. Notice that $u(x) = \sin(2\pi x)$ is the exact solution to the problem. Fig. 1 of the manuscript summarizes the results corresponding to: 1) Single-fidelity data for f, i.e., $n_1 = 0$ and $n_2 = 3$, and 2) Multi-fidelity data for f_1 and f_2, i.e., $n_1 = 15$ and $n_2 = 3$, respectively. The computed values for the optimal model hyper-parameters and parameters for this benchmark example are given in Table 1 for the single-fidelity case and in Table 2 for the multi-fidelity case, respectively.

Fig. 1 highlights the ability of the proposed methodology to accurately approximate the solution to a one dimensional integro-differential equation (see Fig. 1) in the absence of any numerical discretization of the linear operator, or any data on u other than the minimal set of anchor points that are necessary to pin down a solution. In sharp contrast to classical grid-based solution strategies (e.g. finite difference, finite element methods, etc.), our machine learning approach relaxes the traditional well-possedness requirements as the anchor point(s) need not necessarily be prescribed as initial/boundary conditions, and could also be contaminated by noise. Moreover, we see in Fig. 1(C), (E) that a direct consequence of our Bayesian approach is the built-in uncertainty quantification encoded in the posterior variance of u. The computed variance reveals regions where model predictions are least trusted, thus directly quantifying model inadequacy. This information is very useful in designing a data acquisition plan that can be used to optimally enhance our knowledge about u. Giving rise to an iterative procedure often referred to as active learning [22–24], this observation can be used to efficiently learn solutions to differential equations by adaptively selecting the location of the next sampling point.

4.2. Active learning and a-posteriori error estimates for the 2D Poisson equation

Consider the following differential equation

$$\frac{\partial^2}{\partial {x_1}^2}u(x) + \frac{\partial^2}{\partial {x_2}^2}u(x) = f(x), \tag{18}$$

and a single-fidelity data-set comprising of noise-free observations for the forcing term $f(x) = -2\pi^2\sin(\pi x_1)\sin(\pi x_2)$, along with noise free anchor points generated by the exact solution $u(x) = \sin(\pi x_1)\sin(\pi x_2)$. To demonstrate the concept of active learning we start from an initial data set consisting of 4 randomly sampled observations of f in the unit square,

along with 25 anchor points per domain boundary. The latter can be considered as information that is a-priori known from the problem setup, as for this problem we have considered a noise-free Dirichlet boundary condition. Moreover, this relatively high-number of anchor points allows us to accurately resolve the solution on the domain boundary and focus our attention on the convergence properties of our scheme in the interior domain. Starting from this initial training set, we enter an active learning iteration loop in which a new observation of f is augmented to our training set at each iteration according to the chosen sampling policy. Here, we have chosen the most intuitive sampling criterion, namely adding new observations at the locations for which the posterior variance of f is the highest. Despite its simplicity, this choice yields a fast convergence rate, and returns an accurate prediction for the solution u after just a handful of iterations (see Fig. 2(A)). Interestingly, the error in the solution seems to be bounded by the approximation error in the forcing term, except for the late iteration stages where the error is dominated by how well we approximate the solution on the boundary. This indicates that in order to further reduce the relative error in u, more anchor points on the boundary are needed. Overall, the non-monotonic error decrease observed in Fig. 2(A) is a common feature of active learning approaches as the algorithm needs to explore and gather the sufficient information required in order to further reduce the error. Lastly, note how uncertainty in computation is quantified by the computed posterior variance that can be interpreted as a type of *a-posteriori* error estimate (see Fig. 2(C, D)).

4.3. Generality and scalability of the method

It is important to emphasize that as long as the equations are linear, the observations made so far are not problem specific. In fact, the proposed algorithm provides an entirely *agnostic* treatment of linear operators, which can be of fundamentally different nature. For example, we can seamlessly learn solutions to integro-differential, time-dependent, high-dimensional, or even fractional equations. This generality and scalability will be illustrated through a mosaic of benchmark problems compiled in Fig. 3.

4.3.1. Time-dependent linear advection–diffusion–reaction equation

This example is chosen to highlight the capability of the proposed framework to handle time-dependent problems using only noisy scattered space–time observations of the right-hand-side forcing term. To illustrate this capability we consider a time-dependent advection–diffusion–reaction equation

$$\frac{\partial}{\partial t}u(t,x) + \frac{\partial}{\partial x}u(t,x) - \frac{\partial^2}{\partial x^2}u(t,x) - u(t,x) = f(x). \tag{19}$$

Here, we generate a total of $n_1 = 30$ low-fidelity and $n_2 = 10$ high-fidelity training points $(\boldsymbol{t}_1, \boldsymbol{x}_1)$ and $(\boldsymbol{t}_2, \boldsymbol{x}_2)$, respectively, in the domain $[0,1]^2 = \{(t,x) : t \in [0,1] \text{ and } x \in [0,1]\}$. These points are chosen at random according to a uniform distribution. The low- and high-fidelity training data $\{(\boldsymbol{t}_1, \boldsymbol{x}_1), \boldsymbol{y}_1\}, \{(\boldsymbol{t}_2, \boldsymbol{x}_2), \boldsymbol{y}_2\}$ are given by $\boldsymbol{y}_\ell = f_\ell(\boldsymbol{t}_\ell, \boldsymbol{x}_\ell) + \boldsymbol{\epsilon}_\ell$, $\ell = 1, 2$, where $f_2(t,x) = e^{-t}\left(2\pi\cos(2\pi x) + 2(2\pi^2 - 1)\sin(2\pi)x\right)$, and $f_1(t,x) = 0.8 f_2(t,x) - 5tx - 20$. Moreover, $\boldsymbol{\epsilon}_1 \sim \mathcal{N}(0, 0.3\ I)$ and $\boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, 0.05\ I)$. We choose $n_0 = 10$ random anchor points $(\boldsymbol{t}_0, \boldsymbol{x}_0)$ according to a uniform distribution on the initial/boundary set $\{0\} \times [0,1] \cup [0,1] \times \{0,1\}$. Moreover, $\boldsymbol{y}_0 = u(\boldsymbol{t}_0, \boldsymbol{x}_0) + \boldsymbol{\epsilon}_0$ with $\boldsymbol{\epsilon}_0 \sim \mathcal{N}(0, 0.01\ I)$. Note that $u(t,x) = e^{-t}\sin(\pi x)$ is the exact solution. The computed values for the optimal model hyper-parameters and parameters for this benchmark are given in Table 3. Remarkably, the proposed method circumvents the need for temporal discretization, and is essentially immune to any restrictions arising due to time-stepping, e.g., the fundamental consistency and stability issues in classical numerical analysis. As shown in Fig. 3(A), a reasonable reconstruction of the solution field u can be achieved using only 10 noisy high-fidelity observations of the forcing term f (see Fig. 3(A-1, A-2)). More importantly, the maximum error in the prediction is quantified by the posterior variance (see Fig. 3(A-3)), which, in turn, is in good agreement with the maximum absolute point-wise error between the predicted and exact solution for u (see Fig. 3(A-4)). Note that in realistic scenarios no knowledge of the exact solution is available, and therefore one cannot assess model accuracy or inadequacy. The merits of our Bayesian approach are evident – using a very limited number of noisy high-fidelity observations of f we are able to compute a reasonably accurate solution u avoiding any numerical discretization of the spatio-temporal advection–diffusion–reaction operator.

4.3.2. Poisson equation in 10D

To demonstrate scalability to high dimensions, next we consider a 10-dimensional (10D) Poisson equation (see Fig. 3(B)) for which only two dimensions are active in the variability of its solution, namely dimensions 1 and 3. To this end, consider the following differential equation

$$\sum_{d=1}^{10} \frac{\partial^2}{\partial x_d^2} u(x) = f(x). \tag{20}$$

We assume that the low- and high-fidelity data $\{\boldsymbol{x}_1, \boldsymbol{y}_1\}$, $\{\boldsymbol{x}_2, \boldsymbol{y}_2\}$ are generated according to $\boldsymbol{y}_\ell = f_\ell(\boldsymbol{x}_\ell) + \boldsymbol{\epsilon}_\ell$, $\ell = 1, 2$, where $\boldsymbol{\epsilon}_1 \sim \mathcal{N}(0, 0.3\ I)$ and $\boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, 0.05\ I)$. We construct a training set consisting of $n_1 = 60$ low-fidelity and $n_2 = 20$ high-fidelity observations, sampled at random in the unit hyper-cube $[0,1]^{10}$. Moreover, we employ $n_0 = 40$

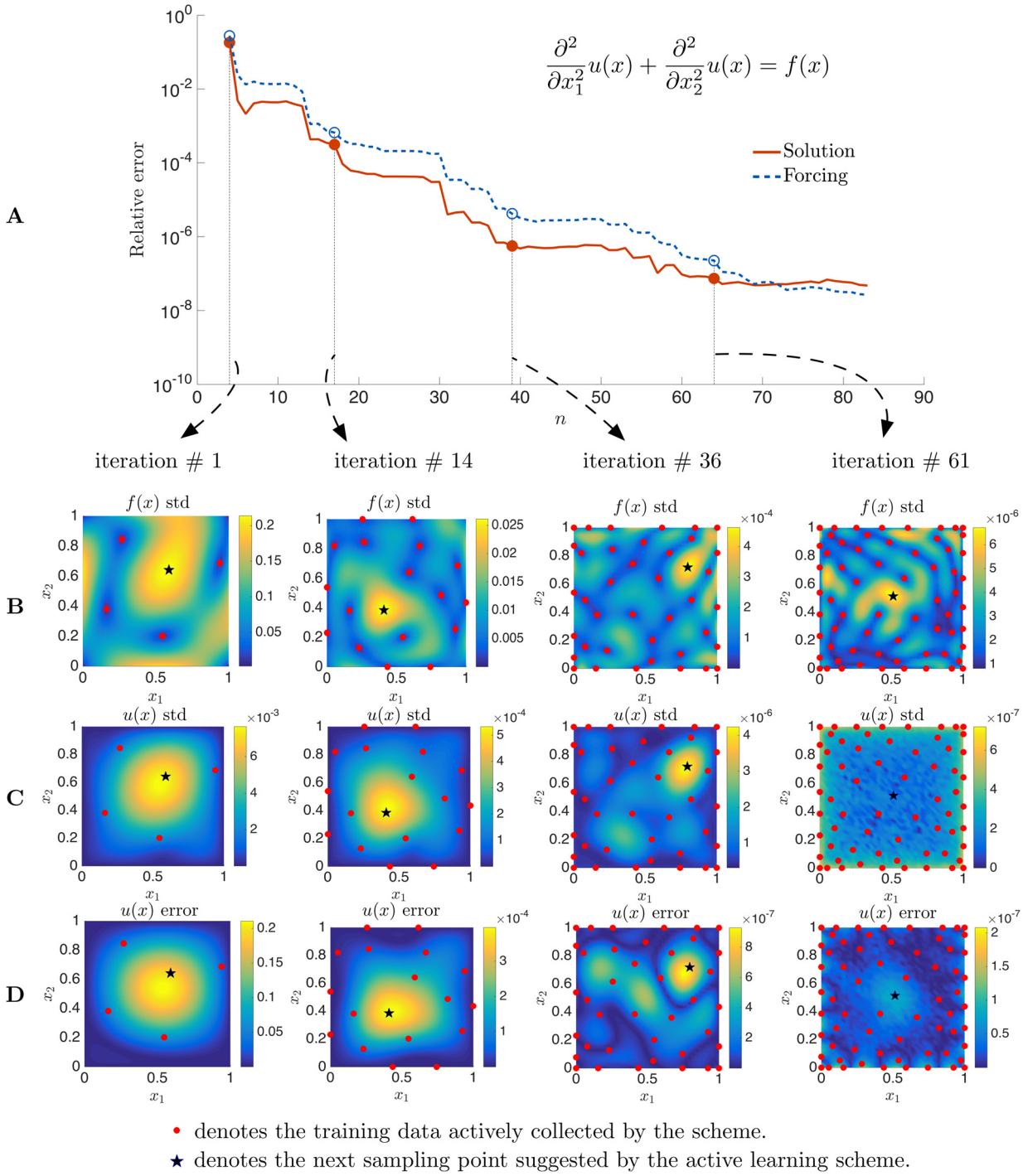

Fig. 2. *Active learning of solutions to linear differential equations and a-posteriori error estimates*: (A) Log-scale convergence plot of the relative error in the predicted solution u and forcing term f as the number of single-fidelity training data on f is increased via active learning. Our point selection policy is guided by the maximum posterior uncertainty on f. (B) Evolution of the posterior standard deviation of f as the number of active learning iterations is increased. (C), (D) Evolution of the posterior standard deviation of u and the relative point-wise error against the exact solution. A visual inspection demonstrates the ability of the proposed methodology to provide an *a-posteriori error estimate* on the predicted solution. Movie S1 presents a real-time animation of the active learning loop and corresponding convergence.

data points on the solution $u(x)$. These anchor points are not necessarily boundary points and are in fact randomly chosen in the domain $[0,1]^{10}$ according to a uniform distribution. The high- and low-fidelity forcing terms are given by $f_2(x) = -8\pi^2 \sin(2\pi x_1)\sin(2\pi x_3)$, and $f_1(x) = 0.8 f_2(x) - 40\prod_{d=1}^{10} x_d + 30$, respectively. Once again, the data $\boldsymbol{y}_0$ on the ex-

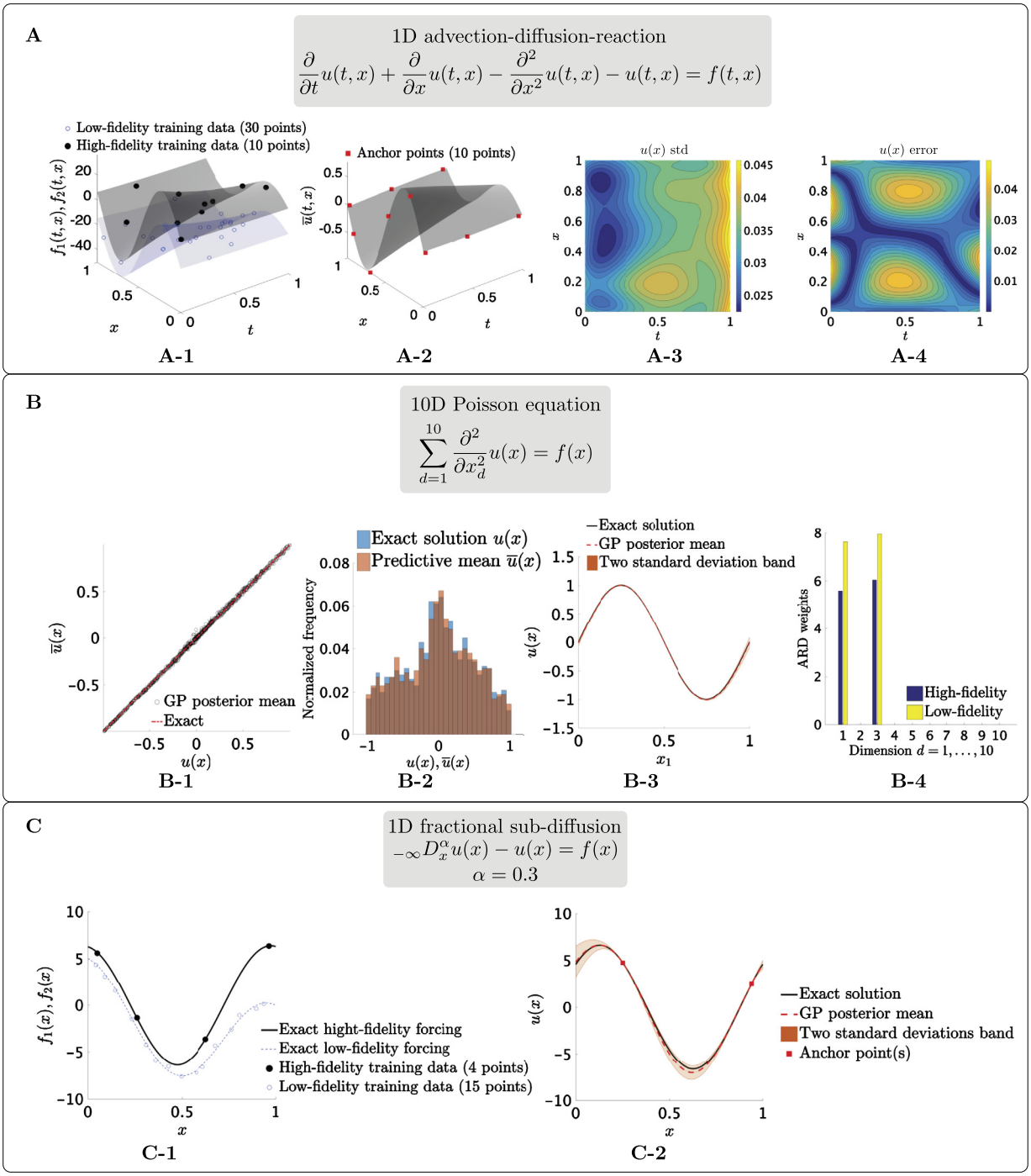

Fig. 3. *Generality and scalability of the multi-fidelity learning scheme*: Equations, variable fidelity data, and inferred solutions for a diverse collection of benchmark problems. In all cases, the algorithm provides an agnostic treatment of temporal integration, high-dimensionality, and non-local interactions, without requiring any modification of the workflow. Comparison between the inferred and exact solutions $\overline{u}$ and u, respectively, for (A) time-dependent advection–diffusion–reaction, (B) Poisson equation in ten dimensions, and (C) fractional sub-diffusion.

act solution $u(x) = \sin(2\pi x_1)\sin(2\pi x_3)$ are generated by $\boldsymbol{y}_0 = u(\boldsymbol{x}_0) + \boldsymbol{\epsilon}_0$ with $\boldsymbol{\epsilon}_0 \sim \mathcal{N}(0, 0.01\ I)$. It should be emphasized that the effective dimensionality of this problem is 2, and the active dimensions x_1 and x_3 will be automatically discovered by our method. In particular, the computed values for the optimal model hyper-parameters and parameters for this benchmark are given in Table 4.

Table 3
Time-dependent linear advection–diffusion–reaction equation: Optimized hyper-parameters and parameters – Multi-fidelity case (i.e., $L = 2$).

σ_2^2	$w_{1,2}$	$w_{2,2}$	$\sigma_{n_2}^2$	$\sigma_{n_0}^2$
1.1593	0.29173	8.327	2.527e−06	0.0019734
σ_1^2	$w_{1,1}$	$w_{2,1}$	$\sigma_{n_1}^2$	ρ
59.164	0.26341	3.3043	0.07568	0.086378

Table 4
Poisson equation in 10D: Optimized hyper-parameters and parameters – Multi-fidelity case (i.e., $L = 2$).

σ_2^2	$w_{1,2}$	$w_{2,2}$	$w_{3,2}$	$w_{4,2}$	$w_{5,2}$	
1.5195	7.7975	1.0199e−06	8.0433	1.4295e−06	2.1383e−07	
$w_{6,2}$	$w_{7,2}$	$w_{8,2}$	$w_{9,2}$	$w_{10,2}$	$\sigma_{n_2}^2$	$\sigma_{n_0}^2$
0.00017021	3.9744e−06	5.8979e−05	5.3126e−07	3.9426e−07	0.00068858	5.1161e−05
σ_1^2	$w_{1,1}$	$w_{2,1}$	$w_{3,1}$	$w_{4,1}$	$w_{5,1}$	
14.36	5.6476	1.0387e−05	6.0426	8.0275e−05	4.7499e−08	
$w_{6,1}$	$w_{7,1}$	$w_{8,1}$	$w_{9,1}$	$w_{10,1}$	$\sigma_{n_1}^2$	ρ
2.3447e−05	7.8975e−05	1.6467e−06	9.5703e−08	4.6577e−05	0.024971	0.52177

Our goal here is to highlight an important feature of the proposed methodology, namely automatic discovery of this *effective dimensionality* from data. This screening procedure is implicitly carried out during model training by using GP covariance kernels that can detect directional anisotropy in multi-fidelity data, helping the algorithm to automatically detect and exploit any low-dimensional structure. Although the high-fidelity forcing f_2 only contains terms involving dimensions 1 and 3, the low-fidelity model f_1 is active along all dimensions. Fig. 3(B-1, B-2, B-3) provides a visual assessment of the high accuracy attained by the predictive mean in approximating the exact solution u evaluated at randomly chosen validation points in the 10-dimensional space. Specifically, Fig. 3(B-1) is a scatter plot of the predictive mean, Fig. 3(B-2) depicts the histogram of the predicted solution values versus the exact solution, and Fig. 3(B-3) is a one dimensional slice of the solution field. If all the dimensions are active, achieving this accuracy level would clearly require a larger number of multi-fidelity training data. However, the important point here is that our algorithm can discover the effective dimensionality of the system from data (see Fig. 3(B-4)), which is a non-trivial problem.

4.3.3. Fractional sub-diffusion equation

Our last example summarized in Fig. 3(C) involves a linear equation with fractional-order derivatives. Such operators often arise in modeling anomalous transport, and their non-local nature poses serious computational challenges as it involves costly convolution operations for resolving the underlying non-Markovian dynamics [25]. Bypassing the need for numerical discretization, our regression approach overcomes these computational bottlenecks, and can seamlessly handle all such linear cases without any modifications. To illustrate this, consider the following one dimensional fractional equation

$$ {}_{-\infty}D_x^{\alpha} u(x) - u(x) = f(x), \tag{21}$$

where $\alpha \in \mathbb{R}$ is the fractional order of the operator that is defined in the Riemann–Liouville sense [25]. In our framework, the only technicality introduced by the fractional operators has to do with deriving the kernel $k(x, x'; \theta)$. Here, $k(x, x'; \theta)$ was obtained by taking the inverse Fourier transform [25]

$$ [(-iw)^{\alpha}(-iw')^{\alpha} - (-iw)^{\alpha} - (-iw')^{\alpha} + 1]\hat{g}(w, w'; \theta), \tag{22}$$

where $\hat{g}(w, w'; \theta)$ is the Fourier transform of the kernel $g(x, x'; \theta)$. Let us now assume that the low- and high-fidelity data $\{\boldsymbol{x}_1, \boldsymbol{y}_1\}, \{\boldsymbol{x}_2, \boldsymbol{y}_2\}$ are generated according to $\boldsymbol{y}_\ell = f_\ell(\boldsymbol{x}_\ell) + \boldsymbol{\epsilon}_\ell$ where $\ell = 1, 2$, $\boldsymbol{\epsilon}_1 \sim \mathcal{N}(0, 0.3\ I)$, $\boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, 0.05\ I)$, $f_2(x) = 2\pi \cos(2\pi x) - \sin(2\pi x)$, and $f_1(x) = 0.8 f_2(x) - 5x$. The training data $\boldsymbol{x}_1, \boldsymbol{x}_2$ with sample sizes $n_1 = 15, n_2 = 4$, respectively, are randomly chosen in the interval $[0, 1]$ according to a uniform distribution. We also assume that we have access to data $\{\boldsymbol{x}_0, \boldsymbol{y}_0\}$ on $u(x)$. In this example, we choose $n_0 = 2$ random points in the interval $[0, 1]$ to define $\boldsymbol{x}_0$ and let $\boldsymbol{y}_0 = u(\boldsymbol{x}_0)$. Notice that

$$ u(x) = \frac{1}{2} e^{-2i\pi x} \left(\frac{-i + 2\pi}{-1 + (-2i\pi)^{\alpha}} + \frac{e^{4i\pi x}(i + 2\pi)}{-1 + (2i\pi)^{\alpha}} \right), \tag{23}$$

is the exact solution, and is obtained using Fourier analysis. Our numerical demonstration corresponds to $\alpha = 0.3$, and our results are summarized in Fig. 3(C). The computed values for the optimal model parameters and hyper-parameters for this benchmark are given in Table 5.

Table 5
Fractional sub-diffusion equation: Optimized hyper-parameters and parameters – Multi-fidelity case (i.e., $L = 2$).

σ_2^2	$w_{1,2}$	$\sigma_{n_2}^2$	$\sigma_{n_0}^2$
63.263	1.1498	0.0074638	2.3315e−05
σ_1^2	$w_{1,1}$	$\sigma_{n_1}^2$	ρ
50.309	8.521	0.1036	1.3371

5. Discussion

In summary, we have presented a probabilistic regression framework for learning solutions to general linear integro-differential equations from noisy data. Our machine learning approach can seamlessly handle spatio-temporal as well as high-dimensional problems. The proposed algorithms can learn from scattered noisy data of variable fidelity, and return solution fields with quantified uncertainty. This methodology generalizes well beyond the benchmark cases presented here. For example, it is straightforward to address problems with more than two levels of fidelity, variable coefficients, complex geometries, non-Gaussian and input-dependent noise models (e.g., student-t, heteroscedastic, etc. [17]), as well as more general linear boundary conditions, e.g., Neumann, Robin, etc. The current methodology can be readily extended to address applications involving characterization of materials, tomography and electrophysiology, design of effective metamaterials, etc. An equally important direction involves solving systems of linear partial differential equations, which can be addressed using multi-output GP regression [26,27].

Another key aspect of this Bayesian mindset is the choice of the prior. Here, for clarity, we chose to start from the most popular Gaussian process prior available, namely the stationary squared exponential covariance function. This choice limits our approximation capability to sufficiently smooth functions. However, one can leverage recent developments in *deep learning* to construct more general and expressive priors that are able to handle discontinuous and non-stationary response [28,29].

Finally we would like to stress that, despite its generality, the proposed framework does not constitute a universal solution remedy for any differential equation. For example, the most pressing open question is posed by non-linear operators for which assigning GP priors on the solution may not be a reasonable choice. Some specific non-linear equations can be transformed into systems of linear equations – albeit in high-dimensions [30–32] – that can be solved with extensions of the current framework.

Acknowledgements

We gratefully acknowledge support from DARPA grant N66001-15-2-4055. We would also like to thank Dr. Panos Stinis (PNNL) for the stimulating discussions during the early stages of this work.

Appendix A. Computer software

All data and results presented in the manuscript can be accessed and reproduced using the Matlab code provided at: https://www.dropbox.com/sh/zt488tymtmfu6ds/AADE2_Yb2Fz8AGBdsUmBXAyEa?dl=0.

Appendix B. Supplementary material

We have generated an animation corresponding to the convergence properties of active learning procedure (see Fig. 2). Movie S1 contains 5 panels. The smaller top left panel shows the evolution of the computed posterior variance of u, while the smaller top right panel shows the corresponding error against the exact solution. Similarly, the smaller bottom left and bottom right panels contain the posterior variance and corresponding relative error in approximating the forcing term f. To highlight the chosen data acquisition criterion (maximum posterior variance of f) we have used a different color-map to distinguish the computed posterior variance of f. Lastly, the larger plot on the right panel shows the convergence of the relative error for both the solution and the forcing as the number of iterations and training points is increased. Fig. 2 shows some snapshots of this animation.

Supplementary material related to this article can be found online at http://dx.doi.org/10.1016/j.jcp.2017.01.060.

References

[1] D. Mumford, The dawning of the age of stochasticity, in: Mathematics: Frontiers and Perspectives, 2000, pp. 197–218.
[2] Z. Ghahramani, Probabilistic machine learning and artificial intelligence, Nature 521 (7553) (2015) 452–459.
[3] M. Jordan, T. Mitchell, Machine learning: trends, perspectives, and prospects, Science 349 (6245) (2015) 255–260.
[4] P. Diaconis, Bayesian numerical analysis, in: Statistical Decision Theory and Related Topics IV, vol. 1, 1988, pp. 163–175.
[5] H. Poincaré, Calcul des probabilités, Gauthier-Villars, 1912.
[6] P. Hennig, M.A. Osborne, M. Girolami, Probabilistic numerics and uncertainty in computations, Proc. R. Soc. A 471 (2015) 20150142.
[7] H. Owhadi, Bayesian numerical homogenization, Multiscale Model. Simul. 13 (3) (2015) 812–828.

[8] P. Hennig, S. Hauberg, Probabilistic solutions to differential equations and their application to Riemannian statistics, in: AISTATS, 2014, pp. 347–355.
[9] J. Skilling, Bayesian solution of ordinary differential equations, in: Maximum Entropy and Bayesian Methods, Springer, 1992, pp. 23–37.
[10] D. Barber, Y. Wang, Gaussian Processes for Bayesian Estimation in Ordinary Differential Equations, ICML, 2014.
[11] O.A. Chkrebtii, D.A. Campbell, B. Calderhead, M.A. Girolami, et al., Bayesian solution uncertainty quantification for differential equations, Bayesian Anal. 11 (4) (2016) 1239–1267.
[12] Hans Kersting, Philipp Hennig, Active uncertainty calibration in Bayesian ODE solvers, arXiv preprint, arXiv:1605.03364, 2016.
[13] Thore Graepel, Solving Noisy Linear Operator Equations by Gaussian Processes: Application to Ordinary and Partial Differential Equations, ICML, 2003.
[14] S. Särkkä, Linear operators and stochastic partial differential equations in Gaussian process regression, in: Artificial Neural Networks and Machine Learning, ICANN 2011, Springer, 2011, pp. 151–158.
[15] I. Bilionis, Probabilistic solvers for partial differential equations, arXiv preprint, arXiv:1607.03526, 2016.
[16] Jon Cockayne, et al., Probabilistic meshless methods for partial differential equations and Bayesian inverse problems, arXiv preprint, arXiv:1605.07811, 2016.
[17] C.E. Rasmussen, Gaussian Processes for Machine Learning, MIT Press, 2006.
[18] K.P. Murphy, Machine Learning: A Probabilistic Perspective, MIT Press, 2012.
[19] M.C. Kennedy, A. O'Hagan, Predicting the output from a complex computer code when fast approximations are available, Biometrika 87 (1) (2000) 1–13.
[20] P. Perdikaris, M. Raissi, A. Damianou, N.D. Lawrence, G.E. Karniadakis, Nonlinear information fusion algorithms for data-efficient multi-fidelity modeling, Proc. R. Soc. A (2017), in press.
[21] L. Le Gratiet, Ph.D. thesis, Université Paris-Diderot-Paris VII, 2013.
[22] D.A. Cohn, Z. Ghahramani, M.I. Jordan, Active learning with statistical models, J. Artif. Intell. Res. 4 (1) (1996) 129–145.
[23] A. Krause, C. Guestrin, Nonmyopic active learning of Gaussian processes: an exploration–exploitation approach, in: Proceedings of the 24th International Conference on Machine Learning, ACM, 2007, pp. 449–456.
[24] D.J. MacKay, Information-based objective functions for active data selection, Neural Comput. 4 (4) (1992) 590–604.
[25] I. Podlubny, Fractional Differential Equations: An Introduction to Fractional Derivatives, Fractional Differential Equations, to Methods of Their Solution and Some of Their Applications, vol. 198, Academic Press, 1998.
[26] M.A. Osborne, S.J. Roberts, A. Rogers, S.D. Ramchurn, N.R. Jennings, Towards real-time information processing of sensor network data using computationally efficient multi-output Gaussian processes, in: Proceedings of the 7th International Conference on Information Processing in Sensor Networks, IEEE Computer Society, 2008, pp. 109–120.
[27] M. Alvarez, N.D. Lawrence, Sparse convolved Gaussian processes for multi-output regression, in: Advances in Neural Information Processing Systems, 2009, pp. 57–64.
[28] A. Damianou, Ph.D. thesis, University of Sheffield, 2015.
[29] G.E. Hinton, R.R. Salakhutdinov, Using deep belief nets to learn covariance kernels for Gaussian processes, in: Advances in Neural Information Processing Systems, 2008, pp. 1249–1256.
[30] R. Zwanzig, Ensemble method in the theory of irreversibility, J. Chem. Phys. 33 (5) (1960) 1338–1341.
[31] A.J. Chorin, O.H. Hald, R. Kupferman, Optimal prediction and the Mori–Zwanzig representation of irreversible processes, Proc. Natl. Acad. Sci. 97 (7) (2000) 2968–2973.
[32] S. Denisov, W. Horsthemke, P. Hänggi, Generalized Fokker–Planck equation: derivation and exact solutions, Eur. Phys. J. B 68 (4) (2009) 567–575.
[33] D.C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. 45 (1–3) (1989) 503–528.
[34] E. Snelson, Z. Ghahramani, Sparse Gaussian processes using pseudo-inputs, in: Advances in Neural Information Processing Systems, 2005, pp. 1257–1264.
[35] J. Hensman, N. Fusi, N.D. Lawrence, Gaussian processes for big data, arXiv preprint, arXiv:1309.6835, 2013.